

ECOM EXPERTS

Ecom fullcache

Manual de usuario, técnico y de desarrollo

ecom_fullcache · 08/09/2026

Contenido

 Vulnerabilidades Mitigadas

- 1. Broken Access Control (OWASP A01)
- 2. Identification and Authentication Failures (OWASP A07)
- 3. Security Misconfiguration (OWASP A05)
- 4. Sensitive Data Exposure (OWASP A03/A02)

 Estado de la Auditoría

Descripción

 El Peligro Mortal: "Cache Poisoning" con Datos de Usuario

Escenarios de Riesgo:

 Reglas de Implementación (Protocolo de Seguridad)

- 1. Validación de Contexto (Lado Servidor)
- 2. "Blacklist" de Controladores
- 3. Validación de Cookies a Nivel de Entrada (get)

Implementación en este Módulo

Este documento certifica que el módulo `ecom_fullcache` ha sido auditado y alineado con los estándares de seguridad definidos en los skills de Agentic Coding (`antigravity-awesome-skills`).

Vulnerabilidades Mitigadas

1. Broken Access Control (OWASP A01)

- **Riesgo Identificado:** Servir páginas privadas (Mi Cuenta, Pedidos) a usuarios públicos desde la caché.
- **Solución Implementada:**
 - Método `isRequestSafe()` bloquea explícitamente controladores de acceso restringido (`order` , `history` , `identity`).
 - Verificación temprana de cookies (`id_customer`) bloquea peticiones autenticadas.
- **Referencia Skill:** `top-web-vulnerabilities/SKILL.md` (Phase 6).

2. Identification and Authentication Failures (OWASP A07)

- **Riesgo Identificado (Session Fixation):** Cachear una respuesta que incluye una cabecera `Set-Cookie` , provocando que todos los visitantes compartan la misma sesión.
- **Solución Implementada:**
 - En `set()` , se escanean las cabeceras de respuesta (`headers_list()`). Si existe `Set-Cookie` , se aborta el cacheado.
- **Referencia Skill:** `broken-authentication/SKILL.md` (Phase 7: Session Fixation Testing).

3. Security Misconfiguration (OWASP A05)

- **Riesgo Identificado (Browser Cache Poisoning):** No enviar cabeceras `Cache-Control` adecuadas, permitiendo que navegadores o proxies intermedios guarden versiones incorrectas (ej. un error 404 o una página logueada).
- **Solución Implementada:**
 - **MISS/UNSAFE:** Se fuerza `Cache-Control: no-store, no-cache` para evitar almacenamiento local.
 - **HIT:** Se envía `Cache-Control: public, max-age=TTL` para optimización segura.
- **Referencia Skill:** `top-web-vulnerabilities/SKILL.md` (Phase 10: Browser Cache Poisoning).

4. Sensitive Data Exposure (OWASP A03/A02)

- **Riesgo Identificado:** Fuga de información personal (PII) o carritos de compra ("Mixed Cart").
- **Solución Implementada:**
 - Validación estricta de `Context->cookie->id_cart` . Si existe carrito, **BLOQUEO TOTAL**.
 - Detección heurística de módulos dinámicos: Bloqueo automático de módulos con keywords como `wishlist` , `review` , `loyalty` .

Estado de la Auditoría

El módulo cumple con las directrices de seguridad backend para sistemas de caché. - [x] Validación de Entradas Controladores (Blacklist) - [x] Validación de Estado de Sesión (Cookie Parsing) - [x] Validación de Salidas (Headers Sanity Check) - [x] Gestión Incorrecta de Navegador (Cache-Control Headers)

SKILL: Seguridad en Cache (Full Page Cache)

Descripción

Este skill define las reglas CRÍTICAS para la implementación de sistemas de Full Page Cache (FPC) en entornos de comercio electrónico (PrestaShop). El objetivo principal es prevenir la fuga de información personal (PII) y el secuestro de sesiones.

⚠ El Peligro Mortal: "Cache Poisoning" con Datos de Usuario

Lo más peligroso de usar cache es **almacenar y servir contenido personalizado** como si fuera contenido público estático.

Escenarios de Riesgo:

1. **Fuga de Carrito:** Un usuario A llena su carrito. Si esa página se cachea, el usuario B verá el carrito de A.
2. **Fuga de PII:** Si un usuario A inicia sesión y la cabecera dice "Hola, Juan", y esto se cachea, todos los visitantes verán "Hola, Juan" y potencialmente enlaces a su cuenta.
3. **Secuestro de Sesión (Session Hijacking):** Si se cachean tokens CSRF o IDs de sesión en formularios, un atacante podría usarlos.

✅ Reglas de Implementación (Protocolo de Seguridad)

Para evitar esto, el sistema de cache debe cumplir **estrictamente** las siguientes validaciones antes de GUARDAR (`set`) o SERVIR (`get`) una página.

1. Validación de Contexto (Lado Servidor)

Nunca confiar solo en cookies del lado cliente para decidir si guardar cache. Validar siempre contra el `Context` de PrestaShop.

- IF `Context::getContext()->customer->isLogged()` -> **NO CACHEAR**
- IF `Context::getContext()->cart->products` (Carrito con items) -> **NO CACHEAR**

2. "Blacklist" de Controladores

Nunca cachear controladores dinámicos. Lista mínima obligatoria: * `cart` * `order` (checkout, opc) * `authentication` (login) * `my-account` (y todas sus sub-secciones: `identity`, `address`, `history`, `discount`) * `password` * `search`

3. Validación de Cookies a Nivel de Entrada (`get`)

Para evitar instanciar el núcleo de PrestaShop innecesariamente (performance), se permite una comprobación rápida de cookies en la fase de lectura (`get`), pero debe ser robusta: * Detectar cualquier

cookie de sesión (`PHPSESSID`). * Detectar cookies nativas de PrestaShop (`PrestaShop-XXX`). * Ante la duda -> **MISS** (No servir cache).

Implementación en este Módulo

Estas reglas se han codificado en `src/Service/CacheGenerator.php` método `shouldSkipCache()` .